# Document Loaders & TextLoader — Student Notes

## 1. Quick Revision: Document

Previously, we manually created a LangChain Document:

```
from langchain_core.documents import Document


document = Document(
    page_content="Python is a programming language.",
    metadata={
        "source": "python_notes.txt"
    }
)
```

A Document contains two important things:

```
Document
   |
   ├── page_content
   |      ↓
   |   Actual Content
   |
   └── metadata
         ↓
```

Information About Content

Example metadata:

```
{
   "source": "python_notes.txt"
}
```

But in real applications, we do not want to manually copy file content into every Document. We want LangChain to read the actual file and create Document objects for us.

## 2. Why Do We Need Document Loaders?

Suppose we already have information inside:

Text Files

PDF Files

Web Pages

CSV Files

Other External Sources

We need an organized mechanism to load that information into LangChain.

That mechanism is:

**Document Loader**

**Definition**

A **Document Loader** is a component that reads information from a supported external source and produces LangChain Document objects.

Simple flow:

External Source

↓

Document Loader

↓

LangChain Document Objects

**Main purpose:**

Source Information

↓

Convert

↓

LangChain Document Format

**3. Document vs Document Loader vs LLM**

These three concepts are different.

Document Loader

↓

Reads Information

Document

↓

Represents Information

LLM

↓

May Later Use Relevant Information

to Generate a Response

**Easy memory:**

Loader = READ

Document = REPRESENT

LLM = USE

## 4. What Is TextLoader?

TextLoader is used for plain text files.

**Definition**

TextLoader is a LangChain Document Loader used to read plain text files and represent their contents as Document objects.

Import:

**from langchain_community.document_loaders import TextLoader**

Installation used in these notes:

**pip install langchain_community**

## 5. Document vs TextLoader

| **Concept** | **Purpose** |
|---|---|
| Document | Represents content and metadata |
| TextLoader | Reads text files and produces Document objects |

**Simple:**

Text File

↓

TextLoader

↓

Document

**6. Why TextLoader When Python Already Has open()?**

Normal Python:

```
with open(
    "student_rules.txt",
    "r",
    encoding="utf-8"
) as file:

    content = file.read()
```

What do we get?

**Python String**

But with LangChain:

```
loader = TextLoader(
    "student_rules.txt",
    encoding="utf-8"
)

documents = loader.load()
```

What do we get?

**List of LangChain Document Objects**

So:

**open()**

↓

**String**

whereas:

**TextLoader**

↓

**Document Objects**

**7. Creating a TextLoader**

```
loader = TextLoader(
    file_path="student_rules.txt",
    encoding="utf-8"
)
```

Important parameters:

| Parameter | Meaning |
|---|---|
| file_path | Identifies the file to load |
| encoding | Specifies how the file text is decoded |

**8. What Does .load() Do?**

```
documents = loader.load()
```

**Definition**

.load() reads the source through the loader and returns the loaded information as a **list of Document objects**.

Simple flow:

Text File

↓

TextLoader

↓

.load()

↓

List of Documents

**9. Why Does .load() Return a List?**

Even when one text file normally creates one Document, LangChain still returns a list.

Example:

**documents = loader.load()**

Conceptually:

documents

↓

**[**

**Document(...)**

**]**

Reason:

**Different Loaders**

↓

**May Produce Different Numbers of Documents**

For example:

**TextLoader**

**↓**

**Normally One Document for One Text File**

**PDF Loader**

**↓**

**May Produce Multiple Documents**

Using a list gives a consistent return structure.

**10. Accessing the First Document**

**document = documents[0]**

Then:

```
print(
    document.page_content
)
```

gives the actual text.

And:

```
print(
    document.metadata
```

**)**

gives the metadata.

## 11. Understanding Metadata

TextLoader supplies source metadata.

Example:

**{**

**"source": "student_rules.txt"**

**}**

We can also add our own custom metadata:

**document.metadata[**

**"department"**

**] = "Administration"**

Concept:

**Loader Metadata**

**+**

**Custom Metadata**

**↓**

**Document Metadata**

**12. Why encoding="utf-8"?**

UTF-8 supports:

**English**

**Telugu**

**Hindi**

**Many Other Languages**

**Symbols such as ₹**

So:

**encoding="utf-8"**

is useful for multilingual text files.

**13. Understanding File Paths**

A common pattern is:

```
from pathlib import Path

base_dir = Path(
    __file__
).resolve().parent

file_path = (
```

**base_dir**

**/ "student_rules.txt"**

**)**

This helps us create a path relative to the Python program.

**14. Understanding __file__**

**__file__**

represents the path of the currently running Python file.

Example:

**D:/projects/chatbot/main.py**

**15. Understanding Path(__file__)**

**Path(**

**__file__**

**)**

converts the file path into a Path object.

Example:

**Path(**

**"D:/projects/chatbot/main.py"**

**)**

A Path object makes file-path operations easier.

**16. Understanding .resolve()**

**Path(**

**__file__**

**).resolve()**

returns an absolute resolved path.

Example:

**D:\projects\chatbot\main.py**

**17. Understanding .parent**

**Path(**

**__file__**

**).resolve().parent**

returns the folder containing the Python file.

Example:

**D:\projects\chatbot**

So:

**__file__**

↓

**Current Python File**

**Path(__file__)**

↓

**Path Object**

**.resolve()**

↓

**Absolute Path**

**.parent**

↓

**Containing Folder**

**18. Complete Path Example**

```
from pathlib import Path

base_dir = Path(
    __file__
).resolve().parent

file_path = (
    base_dir
    / "student_rules.txt"
)

print(
    file_path
)
```

Possible output:

**E:\lcclasses\student_rules.txt**

**19. load() vs lazy_load()**

There are two important loading approaches:

**loader.load()**

and:

**loader.lazy_load()**

**20. What Is lazy_load()?**

lazy_load() provides an iterator that yields Document objects as they become available.

Example:

```
for document in loader.lazy_load():

    print(
        document.page_content
    )
```

## 21. load() vs lazy_load()

| **load()** | **lazy_load()** |
|---|---|
| Returns a list of Documents | Returns an iterator of Documents |
| Collects results before returning | Allows processing as Documents are yielded |
| Convenient for ordinary loading | Useful for iterative processing |

Easy memory:

**load()**

↓

**Give Me the Complete List**

**lazy_load()**

↓

**Give Documents One by One**

## 22. Important Syntax Revision

| Requirement | Syntax |
|---|---|
| Import TextLoader | from langchain_community.document_loaders import TextLoader |
| Create loader | TextLoader(file_path, encoding="utf-8") |
| Load file | loader.load() |
| Iterate lazily | loader.lazy_load() |
| First Document | documents[0] |
| Actual text | documents[0].page_content |
| Metadata | documents[0].metadata |
| Source | documents[0].metadata["source"] |

## 23. Program 1 — Our First TextLoader Program

```
from pathlib import Path


from langchain_community.document_loaders import (
    TextLoader
)
```

```
# ========================================
# CREATE SAMPLE TEXT FILE
# ========================================

file_path = (
    Path(__file__).resolve().parent
    / "student_rules.txt"
)


file_path.write_text(
    "Students must carry their ID cards.\n"
    "Classes start at 9 AM.\n"
    "Mobile phones are not allowed.\n",
    encoding="utf-8"
)


# ========================================
```

```
# CREATE TEXT LOADER
# ========================================

loader = TextLoader(
    file_path=str(file_path),
    encoding="utf-8"
)



# ========================================
# LOAD DOCUMENTS
# ========================================

documents = loader.load()



# ========================================
# DISPLAY TOTAL DOCUMENTS
# ========================================
```

```
print("TOTAL DOCUMENTS")
print("---------------")

print(
    len(documents)
)



# ========================================
# ACCESS FIRST DOCUMENT
# ========================================

document = documents[0]



# ========================================
# DISPLAY CONTENT
# ========================================

print("\nPAGE CONTENT")
```

```
print("------------")

print(
    document.page_content.strip()
)



# ========================================
# DISPLAY SOURCE
# ========================================

print("\nSOURCE FILE")
print("-----------")

print(
    Path(
        document.metadata["source"]
    ).name
)
```

**24. Program 1 Flow**

**Create student_rules.txt**

↓

**Create TextLoader**

↓

**loader.load()**

↓

**List of Documents**

↓

**documents[0]**

↓

**page_content**

**+**

**metadata**

**25. Important Observation**

**documents = loader.load()**

returns:

**List**

Then:

**document = documents[0]**

returns:

**Document**

So:

**documents**

**↓**

**List**

**documents[0]**

**↓**

**Document**

## 26. Program 2 — Understanding Return Type and Metadata

```
from pathlib import Path

from langchain_community.document_loaders import (
    TextLoader
)
```

```
# ========================================
# CREATE SAMPLE TEXT FILE
# ========================================

file_path = (
    Path(__file__).resolve().parent
    / "library_rules.txt"
)


content = (
    "The library opens at 8 AM.\n"
    "The library closes at 6 PM.\n"
)


file_path.write_text(
    content,
    encoding="utf-8"
```

```
)


# ========================================
# CREATE TEXT LOADER
# ========================================

loader = TextLoader(
    file_path=str(file_path),
    encoding="utf-8"
)


# ========================================
# LOAD DOCUMENTS
# ========================================

documents = loader.load()
```

```
# ========================================
# CHECK RETURN TYPE
# ========================================

print("RETURN TYPE")
print("-----------")

print(
   type(documents).__name__
)



# ========================================
# CHECK TOTAL DOCUMENTS
# ========================================

print("\nTOTAL DOCUMENTS")
print("---------------")

print(
```

```
    len(documents)
)



# ========================================
# ACCESS DOCUMENT
# ========================================

document = documents[0]



print("\nDOCUMENT TYPE")
print("-------------")

print(
    type(document).__name__
)



# ========================================
```

```
# DISPLAY SOURCE METADATA
# ========================================

print("\nSOURCE FILE")
print("-----------")

print(
    Path(
        document.metadata["source"]
    ).name
)



# ========================================
# ADD CUSTOM METADATA
# ========================================

document.metadata[
    "department"
] = "Library"
```

```
# ========================================
# DISPLAY CUSTOM METADATA
# ========================================

print("\nDEPARTMENT")
print("----------")

print(
   document.metadata[
      "department"
   ]
)



# ========================================
# CHECK ORIGINAL FILE
# ========================================
```

```
print("\nORIGINAL FILE UNCHANGED?")
print("-----------------------")

print(
    file_path.read_text(
        encoding="utf-8"
    )
    == content
)
```

## 27. Important Learning from Program 2

The important conclusion is:

TextLoader creates Document objects with source metadata. We can extend the metadata without changing the original text file.

Flow:

**Original Text File**

↓

**TextLoader**

↓

**Document**

↓

Add Custom Metadata

↓

Original Text File Remains Unchanged

28. Program 3 — Loading Multiple Text Files

```
from pathlib import Path

from langchain_community.document_loaders import (
    TextLoader
)



# ========================================
# CREATE DEMO FOLDER
# ========================================

base_dir = (
    Path(__file__).resolve().parent
    / "day2_notices"
```

```
)


base_dir.mkdir(
    exist_ok=True
)


# ========================================
# CREATE THREE TEXT FILES
# ========================================

file_data = {

    "student_rules.txt":
        "Students must carry their ID cards.",

    "library_notice.txt":
        "The library closes at 6 PM.",
```

```
    "fee_notice.txt":
        "Course fee deadline is 15 November 2026."
}



for file_name, content in file_data.items():

    file_path = (
        base_dir
        / file_name
    )

    file_path.write_text(
        content,
        encoding="utf-8"
    )



# ========================================
# LOAD ALL TEXT FILES
```

```
# ========================================

all_documents = []



for file_name in file_data:

    file_path = (
        base_dir
        / file_name
    )

    loader = TextLoader(
        file_path=str(file_path),
        encoding="utf-8"
    )

    documents = loader.load()

    all_documents.extend(
```

```
        documents
    )


# ========================================
# DISPLAY TOTAL DOCUMENTS
# ========================================

print("TOTAL DOCUMENTS")
print("---------------")

print(
    len(all_documents)
)


# ========================================
# DISPLAY EACH DOCUMENT
# ========================================
```

```
for number, document in enumerate(
    all_documents,
    start=1
):

    print(
        "\nDOCUMENT",
        number
    )

    print(
        "SOURCE:",
        Path(
            document.metadata["source"]
        ).name
    )

    print(
        "CONTENT:",
        document.page_content
```

)

**29. Multiple Files Flow**

**student_rules.txt**

**library_notice.txt**

**fee_notice.txt**

↓

**Loop**

↓

**Create TextLoader**

**for Each File**

↓

**loader.load()**

↓

**Documents**

↓

**all_documents**

**30. Why extend()?**

Each call:

**documents = loader.load()**

returns a list.

Suppose:

**documents = [**

**Document(...)**

**]**

We want to add each Document into:

**all_documents**

So we use:

**all_documents.extend(**

**documents**

**)**

Final structure:

**all_documents**

**↓**

**[**

**Document 1,**

**Document 2,**

**Document 3**

**]**

## 31. Main Conclusion — Multiple Files

We can use TextLoader repeatedly to convert information from multiple text files into a collection of LangChain Document objects.

## 32. Program — Handling a Missing Text File

Document loading requires the actual source file to exist and be accessible.

```
from pathlib import Path


from langchain_community.document_loaders import (
    TextLoader
)



# ========================================
# SPECIFY NON-EXISTING FILE
# ========================================
```

```
file_path = (
    Path(__file__).resolve().parent
    / "missing_notice_for_demo_98765.txt"
)



# ========================================
# CREATE TEXT LOADER
# ========================================

loader = TextLoader(
    file_path=str(file_path),
    encoding="utf-8"
)



# ========================================
# ATTEMPT TO LOAD FILE
# ========================================
```

```
try:

    documents = loader.load()

    print(
        "TOTAL DOCUMENTS:",
        len(documents)
    )


except RuntimeError:

    print(
        "FILE LOADING FAILED"
    )

    print(
        "The file could not be read."
    )
```

```
print(
    "Check the file path,"
    " permissions and encoding."
)
```

**33. Why Can Loading Fail?**

Possible things to check:

**File Path**

**Permissions**

**Encoding**

**Actual File Availability**

Main conclusion:

**Document loading depends on the availability and accessibility of the actual source file.**

**34. Program — Understanding lazy_load()**

```
from pathlib import Path

from langchain_community.document_loaders import (
    TextLoader
)



# ========================================
# CREATE SAMPLE FILE
# ========================================

file_path = (
    Path(__file__).resolve().parent
    / "course_notice.txt"
)



file_path.write_text(
    "New Python batch starts on Monday.\n"
```

```
    "Class timing is 7 AM.\n",
    encoding="utf-8"
)



# ========================================
# CREATE TEXT LOADER
# ========================================

loader = TextLoader(
    file_path=str(file_path),
    encoding="utf-8"
)



# ========================================
# PROCESS DOCUMENTS LAZILY
# ========================================

count = 0
```

```
for document in loader.lazy_load():

    count += 1

    print(
        "DOCUMENT",
        count
    )

    print(
        "CONTENT:",
        document.page_content.strip()
    )

    print(
        "SOURCE:",
        Path(
            document.metadata["source"]
```

```
        ).name
    )



# ========================================
# DISPLAY TOTAL
# ========================================

print(
    "\nTOTAL PROCESSED:",
    count
)
```

**35. lazy_load() Flow**

**Text File**

**↓**

**TextLoader**

**↓**

**lazy_load()**

**↓**

**Yield Document**

↓

**Process It**

↓

**Yield Next Document**

↓

**Process It**

Main conclusion:

**load() collects Document objects into a list, whereas lazy_load() allows us to process Document objects as they are yielded.**

**36. Most Important Comparison**

**NORMAL PYTHON**

**==========**

**Text File**

↓

**open()**

↓

**String**

**LANGCHAIN**

**=========**

**Text File**

**↓**

**TextLoader**

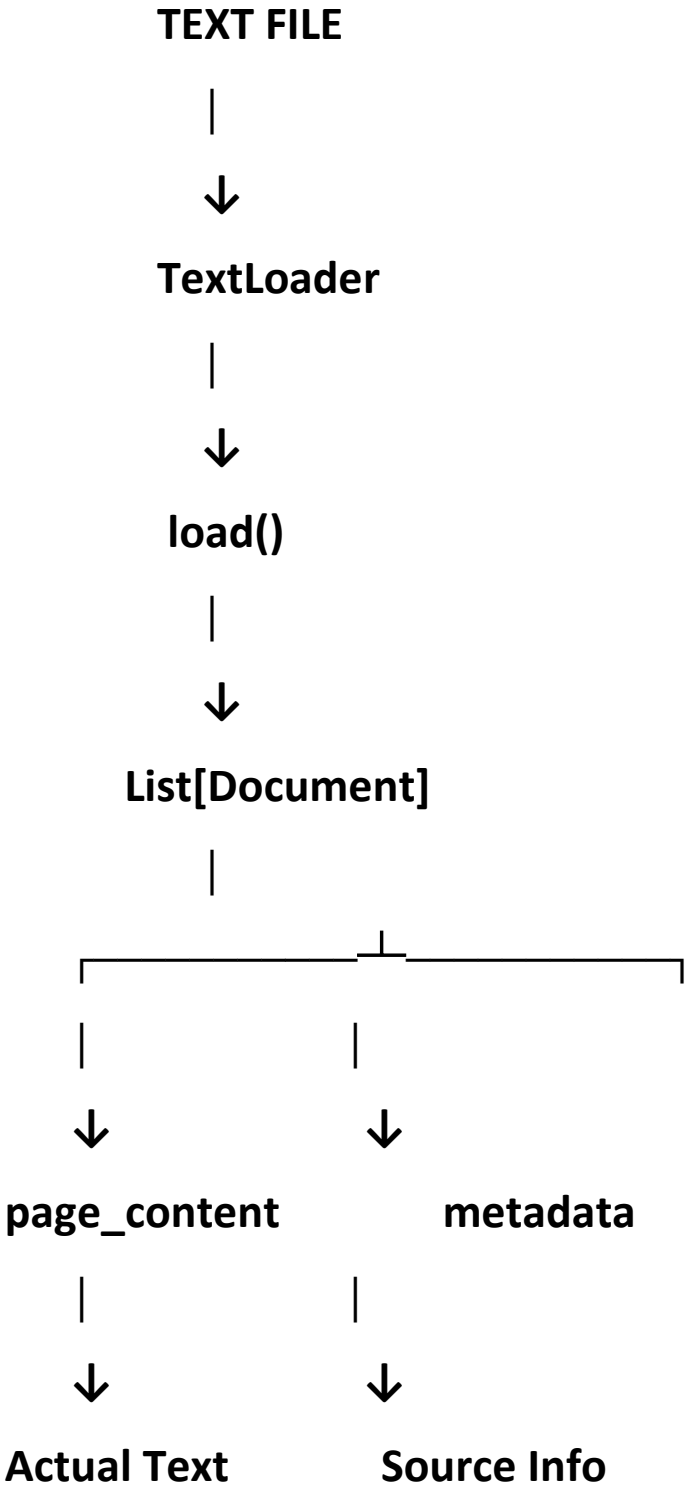
**↓**

**Document**

**↓**

**page_content + metadata**

## 37. Complete TextLoader Architecture



## 38. Important Key Points

1. A Document represents content and metadata.
2. A Document Loader reads information from an external source.
3. TextLoader is used for plain text files.
4. TextLoader produces LangChain Document objects.
5. .load() returns a list of Document objects.
6. document.page_content gives the actual text.
7. document.metadata gives metadata.
8. metadata["source"] identifies the source file.
9. We can add custom metadata to a Document.
10. Adding custom metadata does not modify the original text file.
11. UTF-8 supports multiple languages and symbols.
12. Path(__file__).resolve().parent gives the folder containing the Python program.
13. TextLoader can be repeatedly used to load multiple text files.
14. The actual source file must exist and be accessible.
15. .lazy_load() supports iterative Document processing.

**39. Common Mistake — Thinking Loader Is the Document**

Wrong:

**TextLoader = Document**

Correct:

**TextLoader**

**↓**

**Reads Source**

**↓**

**Creates Document**

**40. Common Mistake — Thinking .load() Returns a String**

Wrong:

**documents = loader.load()**

does not normally give:

**String**

It gives:

**List of Document Objects**

## 41. Common Mistake — Accessing page_content Directly from the List

Wrong:

**documents.page_content**

Correct:

**documents[0].page_content**

Because:

**documents**

↓

**List**

**documents[0]**

↓

**Document**

## 42. Common Mistake — Confusing Source File with Metadata

The actual file contains the text.

The Document metadata contains information such as:

{

"source": "student_rules.txt"

}

So:

**File**

**↓**

**Actual Data Source**

**metadata["source"]**

**↓**

**Information About Source**

## 43. Common Mistake — Thinking Custom Metadata Changes the Original File

If we write:

**document.metadata[**

**"department"**

**] = "Library"**

we change the Document metadata.

We do **not** automatically change:

**library_rules.txt**

## 44. Common Mistake — Thinking lazy_load() Returns the Complete List Immediately

lazy_load() is designed for iterative processing.

Example:

```
for document in loader.lazy_load():
    ...
```

Whereas:

```
documents = loader.load()
```

collects the results into a list.

## 45. Student Memory Formula

**SOURCE**

↓

**LOADER**

↓

**DOCUMENT**

↓

**CONTENT + METADATA**

**For our topic:**

**.txt File**

↓

**TextLoader**

↓

**.load()**

↓

**List[Document]**

↓

**page_content**

**+**

**metadata**

## 46. One-Line Conclusion

**TextLoader reads a plain text file and converts its information into LangChain Document objects containing the actual text in page_content and source information in metadata.**